API Reference: Booking Validation

Portfolio Context

The challenge: The core XML schema underpinning the booking platform is massive (over 20,000 lines). Autogenerated documentation from the XSD was too dense and unhelpful for external integrators who only needed a subset of the fields.

My approach: I manually documented the crucial response elements, translating internal engineering terms into clear developer-facing definitions. I used definition lists and nested tables to represent the XML hierarchy clearly.

Note: Elements of the payload and company specifics have been anonymized for this portfolio piece.

`<validateBookingResponse>`

The `validateBookingResponse` message returns the complete state of a Booking Record after a successful payment transaction. It confirms that tickets have been issued and provides the financial settlement details.

This response is extremely comprehensive. The root elements you will primarily interact with are:

- `<bookingRecordLocator>`
- `<passengers>`
- `<travelSegments>`
- `<ticketableFares>`

Root Elements

`<bookingRecordLocator>`

The 32-character UUID that uniquely identifies this transaction in the platform. You must store this ID locally; it is required for all future after-sales operations (e.g., refunds or exchanges).

`<dateBooked>`

The ISO-8601 timestamp representing the exact moment the carrier confirmed the payment and transitioned the booking status to `CONFIRMED`.
Format: `YYYY-MM-DDThh:mm:ss.sssZ`

Understanding `<ticketableFares>`

The `<ticketableFares>` array contains one or more `ticketableFare` objects. This is the financial core of the response, detailing exactly what was charged, by whom, and the resulting ticket IDs.

Within each `<ticketableFare>`, you will find:

Element	Data Type	Description
`<fareId>`	String	A unique identifier for this specific pricing component.
`<passengerIds>`	Array	References the `passengerID` (e.g., `PAX_SPEC_0`) defined earlier in the request, linking this cost to a specific traveler.
`<travelSegmentIds>`	Array	References which segment this fare pays for (e.g., `SEG_0`).
`<fareState>`	String	The fulfillment status. A successful validation will always return `CONFIRMED_AND_PAID`.

Financial breakdown (`<prices>`)

Nested inside the fare block is the `<prices>` object, which handles the complex currency logic required for multi-national operators.

Handling Multiple Currencies

Do not assume `<amountNet>` equals `<amountGross>`. You must inspect the `currencyCode` attribute. If the point-of-sale currency differs from the carrier's native currency, an overarching conversion rate is applied to the gross.

```
<ticketableFare>
   <prices>
      <amountGross currencyCode="GBP">45.50</amountGross>
      <amountNet currencyCode="GBP">45.50</amountNet>
      <!-- Optional tax breakdown -->
      <taxes>
         <tax taxCode="VAT">
            <amount currencyCode="GBP">9.10</amount>
         </tax>
      </taxes>
   </prices>
</ticketableFare>
```

Fulfillment Details (`<fulfillment>`)

Once paid, the carrier issues the actual travel documents. The fulfillment block tells your application how to retrieve or render the tickets.

`deliveryMethod` *(Enum)* : The format the tickets are provided in. Common values: - `E_TICKET` : A binary PDF stream is attached to the response. - `TOD` : Ticket-on-Departure; requires the passenger to use a kiosk at the station. - `MOBILE_BARCODE` : An Aztec/QR code string is provided.

`<ticketReferences>` *(Array)* : Contains the carrier's PNR (Passenger Name Record) or Ticket Number that the passenger will actually use to travel.

```
<fulfillment>
    <deliveryMethod>MOBILE_BARCODE</deliveryMethod>
    <ticketReferences>
        <ticketReference>
            <referenceType>PNR</referenceType>
            <referenceValue>X79QMB</referenceValue>
        </ticketReference>
    </ticketReferences>
</fulfillment>
```